

When Does Adapting a Queue Discipline’s Parameters Pay? A Scaling Law in `target`/RTT, Measured on Stock Linux

Amritha S Yugeshwaran P Deepti Annuncia

Department of Electronics and Communication Engineering

Vellore Institute of Technology, Chennai, India

Abstract

Linux ships `fq_codel` with four parameters fixed at configuration time. Whether adapting them at runtime is worth doing has been answered both ways, and neither answer states when the other holds. Across 178 runs spanning three workloads, eight alternative queue disciplines and path round-trip times from 2 to 200ms, we find the benefit of adaptation is governed not by path RTT but by the dimensionless ratio $r = \text{target}/\text{RTT}$. The benefit follows a logistic curve in r with ceiling 20.5 % and half of that reached at $r = 0.50$ ($R^2 = 0.986$), a form that holds across a 16-fold change in RTT and transfers to `codel` and PIE. Published guidance places a correctly configured deployment at r between 0.05 and 0.10, so half the available benefit lies five to ten times outside that envelope: adaptation is a remedy for misconfiguration rather than an improvement on correct configuration. Two measurements frame the result. Flow queueing with a delay target cuts tail latency 99.0 % below an unmanaged queue at indistinguishable goodput, two orders of magnitude more than any tuning measured here; and CAKE, which needs no controller, beats the adapted system at every RTT tested. All hypotheses and decision thresholds were registered before the data existed.

Keywords: active queue management; bufferbloat; `fq_codel`; CoDel; dimensionless scaling law; runtime parameter adaptation; Linux `tc`; pre-registered evaluation

1 Introduction

Bufferbloat is the inflation of queueing delay by buffers larger than any control loop requires, while throughput stays unaffected[1]. Linux answers it with `fq_codel`: Deficit Round Robin across roughly a thousand flow queues, each with its own CoDel instance[2]. Four parameters govern it and none changes once set: `target` (5 ms), `interval` (100 ms), `limit` and `quantum`.

Adapting those parameters at runtime has clear precedent. Adaptive RED adjusts a drop probability by an AIMD rule[3]; Ye and Leung adapt CoDel’s interval analytically[4]; DESiRED adapts an AQM’s delay target in P4 with deep reinforcement learning[5]. Runtime adaptation of a delay target is therefore not novel and we claim no novelty for it.

What the literature does not supply is the condition under which adaptation pays. Systems report gains; independent evaluations report none; neither camp states

when the other is right. RFC 7567 meanwhile recommends AQM that needs no per-flow configuration[6], which makes a null result for adaptation a reportable outcome rather than a failed experiment.

This paper establishes that condition experimentally, and it turns out to be a dimensionless ratio rather than any absolute quantity. Section 2 places the work against the base paper and six recent systems, Section 3 describes the apparatus, Section 4 gives the measurements, and Section 5 derives and tests the law.

2 Related work

Random Early Detection opened the field, dropping with a probability derived from average queue length[7]; its sensitivity to configuration prompted Adaptive RED[3], whose AIMD policy and $\beta = 0.9$ decrease factor our controller adopts. PIE replaced queue length with a delay estimate inside a PI controller[8]. CoDel abandoned queue length entirely, dropping when minimum sojourn time over a sliding interval exceeds a target[9, 10]; its authors argue the parameters are RTT-relative by construction and need no tuning. Equation 1 quantifies where that argument stops holding. CAKE integrates shaping, flow queueing and host fairness[11], FQ-PIE is the corresponding combination for PIE[12], and L4S standardises a low-latency class on a dual-queue coupled AQM[13, 14]. All eight disciplines are measured here under one identical bottleneck.

Table 1 places this work against the base paper and six recent systems. Two things follow. Adapting a delay target at runtime is established, so the mechanism is not our contribution. And no entry states the condition under which its adaptation pays, which is what we supply.

Table 1: The base paper and six recent systems. None reports the condition under which its adaptation is worth running.

System	Year	Adapts	Method
Adaptive RED[3]	2001	<code>max_p</code>	AIMD
ACoDel[4]	2020	<code>interval</code>	analytical
QueuePilot[15]	2023	marking prob.	offline RL
DualPI2 / L4S[14]	2023	none	dual queue
DESiRED[5]	2024	target delay	deep RL
AQM-LLM[16]	2025	action policy	distilled LLM
This work	2026	four parameters	AIMD + gradient

What none of them runs is a control condition separating a controller’s computational cost from the effect of its decisions, and none reports sparse-probe and bulk-

flow latency separately despite flow queueing privileging sparse flows by construction. Both are needed before a small difference can be attributed to anything.

3 Method

Three network namespaces model a client, a router and a server. The router carries `netem` for base delay and a token-bucket shaper at 10Mbit/s as root `qdisc`, with the discipline under test as its child on the egress *toward the server*, so the bottleneck sits on the data path. An earlier two-node arrangement shaped the reverse direction, and every backlog figure taken that way measured acknowledgements rather than data.

Nine configurations are compared: `pfifo`, `SFQ`, `RED`, `code1`, `PIE`, `FQ-PIE`, `CAKE`, static `fq_code1`, and `fq_code1` under our controller. A tenth, the *sham* controller, polls at the controller’s cadence and runs the same classification and prediction code but applies no change, which separates the controller’s CPU cost from its decisions.

The controller reads `tc` statistics and per-flow state from an eBPF program at the `tc` `clsact` egress hook, classifies a congestion regime, estimates a trajectory by least squares over a ten-sample window, and applies an AIMD adjustment. Observation and update run on separated timescales, an order of magnitude apart, following two-timescale stochastic approximation[17]. Every bound and step size is expressed relative to the configured `target`, not in absolute milliseconds; expressed absolutely, a controller started against an 80ms target would take steps 16 times smaller in relative terms than against 5ms, and the measured effect would then be an artefact of the constants.

Three latencies are measured separately: a sparse ICMP probe, bulk TCP flow RTT reported by `iperf3`, and a queue-delay proxy derived from backlog and rate. Workloads are a steady eight-flow bulk load, a staged load whose flow count varies from two to twenty-four during the run, and a mixed load adding sparse UDP probes. Each cell is repeated three times. Differences use Welch’s *t*-test with exact two-sided *p*-values from the regularised incomplete beta function, since at three samples per group the normal approximation materially overstates significance. The transport is TCP CUBIC[18] on Linux 6.12.48 with every discipline compiled in.

4 Results

Every number below is generated from the committed run logs by the analysis scripts; none is entered by hand.

4.1 The dominant effect is not tuning

Figure 1 gives 95th-percentile probe latency for all nine configurations. The disciplines that combine flow queueing with a delay target sit within a few milliseconds of one another; the unmanaged queue sits two orders of magnitude above, at 2344ms against 24.57ms,

a 99.0% reduction ($p < 0.001$) at goodput that is statistically indistinguishable (9.453 against 9.414Mbit/s, $p = 0.20$). The static, sham and adapted `fq_code1` points are not separable at this scale, which is the first indication that parameter tuning is a second-order effect.

Fair queueing alone is not sufficient. `SFQ` isolates flows but applies no AQM, and its bulk queues sit at their 127-packet limit throughout: its probe reads 30.68ms while the bulk flows sharing the link read 294.8ms. The scheduler and the AQM do distinct jobs and both are necessary.

4.2 What adaptation achieves, parameter by parameter

Figure 2 places every measured quantity against every system. Under the steady workload the controller produces a real but small improvement: mean backlog falls from 6.79 to 6.14 packets. The sham comparison attributes this to the control decisions rather than to the presence of a busy process, since the static-to-sham difference lies inside the confidence interval on every metric. Under the staged workload, the one built to exercise adaptation, no metric separates: adjustments lag the transitions they respond to.

`CAKE` is the result that matters for practice. It requires no controller, no telemetry and no model, and it reaches 22.9ms probe p95 against the adapted system’s 24.43ms. Switching discipline is worth more than tuning one.

5 A scaling law for when adaptation pays

The measurements above are taken at one path RTT. Sweeping base RTT from 2 to 200ms while holding the configured `target` fixed shows the benefit of adaptation is not a function of RTT at all, but of the ratio $r = \text{target}/\text{RTT}$. Fitting bulk-flow RTT reduction b over 10 cells gives

$$b(r) = \frac{B}{1 + (r_0/r)^k}, \quad \begin{array}{l} B = 20.5\% \\ r_0 = 0.50 \\ k = 2.6 \end{array} \quad (1)$$

with $R^2 = 0.986$, where B is the ceiling, r_0 the ratio at which half of it is reached and k the sharpness of the transition (Figure 3).

The hypotheses and their decision thresholds were committed to the repository before the data existed. Three held. A crossover exists: benefit stays under 1.5% and is not significant below $r = 0.25$, and reaches 16.4–20.1% at $r \geq 1$. The curve is invariant across a 16-fold change in RTT: at fixed $r = 1$, cells at 5, 20 and 80ms give 18.4, 17.6 and 16.4%, a spread of 2.0 percentage points against a 5-point threshold. Leave-one-out prediction of the interior ratios has a mean absolute error of 1.4 percentage points, worst case 3.5. The form also transfers to mechanisms the fit never

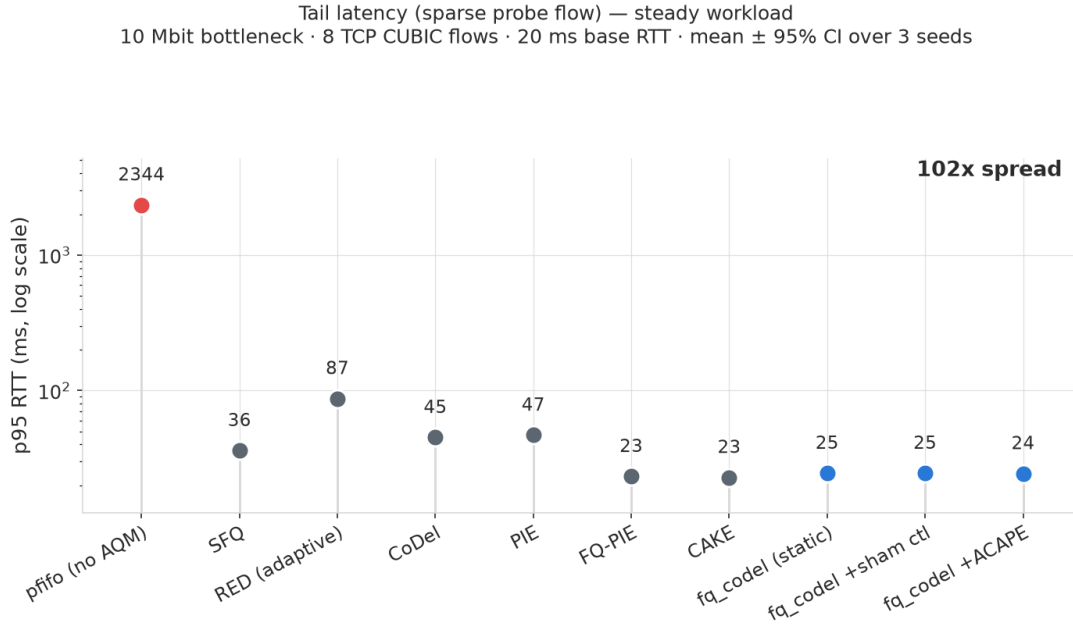


Figure 1: Tail latency across all nine configurations under steady load, logarithmic axis. The dominant effect by two orders of magnitude is the presence of flow queueing with a delay target, not the tuning of its parameters. CAKE, which requires no controller, outperforms the adapted system. Values are means over three runs with 95 % confidence intervals.

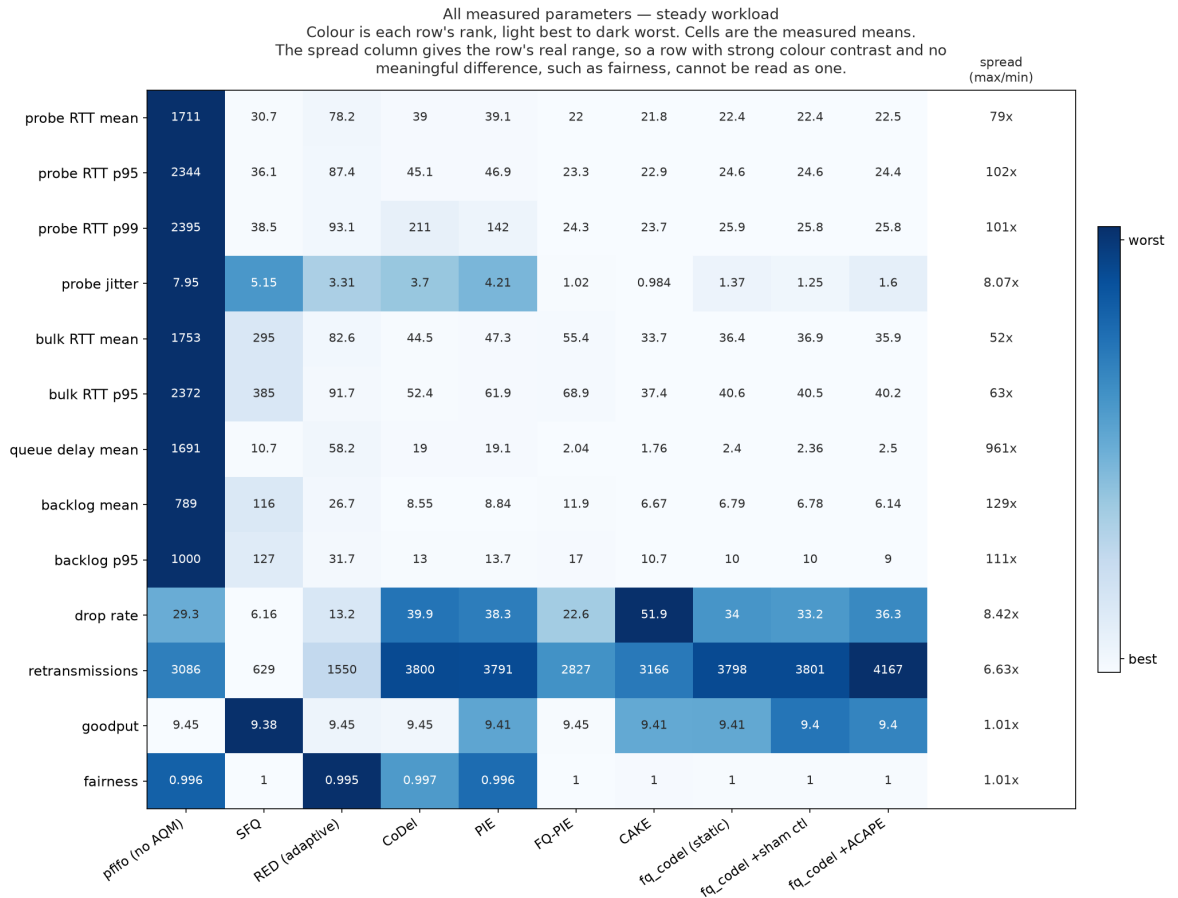
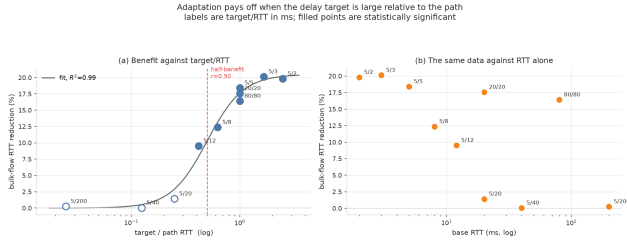


Figure 2: Every measured quantity against every system, steady workload. Colour is each row's rank, light best to dark worst; cells are measured means. The spread column gives each row's real range, so rows with strong colour contrast but no meaningful difference, such as goodput and fairness at 1.01×, are not read as findings.



- [11] T. Høiland-Jørgensen, D. Täht, and J. Morton, “Piece of CAKE: a comprehensive queue management solution for home gateways,” in *Proc. IEEE LANMAN*, 2018.
- [12] G. Ramakrishnan, M. Bhasi, V. Saicharan, L. Monis, S. D. Patil, and M. P. Tahiliani, “FQ-PIE queue discipline in the Linux kernel: design, implementation and challenges,” in *Proc. IEEE LCN*, 2019.
- [13] B. Briscoe, K. De Schepper, M. Bagnulo, and G. White, “Low latency, low loss and scalable throughput (L4S) Internet service: architecture,” IETF, Tech. Rep. RFC 9330, 2023.
- [14] K. De Schepper, B. Briscoe, and G. White, “Dual-queue coupled active queue management for L4S,” IETF, Tech. Rep. RFC 9332, 2023.
- [15] M. Dery, O. Krupnik, and I. Keslassy, “QueuePilot: reviving small buffers with a learned AQM policy,” in *Proc. IEEE INFOCOM*, 2023.
- [16] S. R. Pokhrel, D. Satish, J. Kua, and A. Walid, “Distilling large language models for network active queue management,” *IEEE Trans. Netw.*, 2025.
- [17] V. S. Borkar, “Stochastic approximation with two time scales,” *Syst. Control Lett.*, vol. 29, no. 5, pp. 291–294, 1997.
- [18] S. Ha, I. Rhee, and L. Xu, “CUBIC: a new TCP-friendly high-speed TCP variant,” *ACM SIGOPS Oper. Syst. Rev.*, vol. 42, no. 5, pp. 64–74, 2008.